1K FOLLOWER COMMUNITY CHALLENGE

Sprout Challenge

Rulebook. Code a robot dog through an obstacle course. Fastest robot wins a $20 gift card.

TASK	Write `act(observation) → 12 joint angles`, called 50 times a second
DEADLINE	30 November 2026, 23:59 IST (18:29 UTC)
RESULTS	by 3 December 2026, on Instagram
ENTER	DM a public GitHub repo link to @shantanu.builds.robots
CODE	github.com/shantanu-builds-robots/sprout-challenge



Version of 10 November 2026. The latest rules are always RULES.md in the repository; if this PDF and RULES.md differ, RULES.md applies.

Contents

1. Timeline
2. Who can enter
3. What you submit
4. How to enter
5. A run
6. Barriers
7. Scoring
8. What's allowed
9. Grading
10. Prize
11. Judging and disputes
12. Instagram

Sprout Challenge rulebook

The Sprout Challenge celebrates 1,000 followers on Instagram (@shantanu.builds.robots). Write a controller that gets Sprout, a simulated four-legged robot, from the start line to the finish line of an obstacle course as fast as possible. The fastest robot wins a $20 gift card.

It's a community challenge and a contest of skill: no luck draw, nothing to buy. The **top 10** entries get their runs featured on the account.

Setting up your computer is covered in INSTALL.md; the robot, its sensors and the examples are in the README.

1. Timeline

Opens	the day the announcement reel is posted on Instagram. The practice track and all the tools are public from then.
Deadline	**30 November 2026, 23:59 IST (UTC+5:30)**, which is 18:29 UTC. Late entries are not graded.
Results	by 3 December 2026, on Instagram. The competition arena and the grading seeds are published with the results.

2. Who can enter

- Anyone, anywhere, of any age, unless the law where you live forbids it.
- If you are under 18, a parent or guardian must agree to your entry, and they receive the prize on your behalf.
- You must follow @shantanu.builds.robots on Instagram, and still follow it at the deadline.
- **One entry per person.** A team counts as one person and shares the prize.

3. What you submit

A **submission folder** containing a file named `controller.py` and anything it loads (network weights, lookup tables, helper modules). `controller.py` defines one subclass of `sprout_challenge.Controller`:

```
from sprout_challenge import Controller

class MyController(Controller):
    def reset(self, info):      # once, before each run: the track's layout and the robot's
limits
        self.pose = info["default_pose"]

    def act(self, obs):         # 50 times a second: sensors in, 12 joint angles [rad] out
        return self.pose.copy()
```

- Load your files relative to `__file__`, not the current directory.
- Put the submission folder in a **public GitHub repository**: either as the repository itself (`controller.py` at the top), or as the **only** folder in the repository that contains a `controller.py`.
- Symbolic links, `.pyc` files and `__pycache__` folders are ignored when your entry is collected. Commit the real files.
- The whole submission folder must be under 200 MB.

The `examples/` folder has three valid entries, from one that stands still to a neural network that finishes the practice track.

4. How to enter

1. Put your submission in a public GitHub repository (see section 3).
2. Check it in the grading container (section 9), so you know it runs the way the judges will run it.
3. **DM @shantanu.builds.robots the link to the repository.**

You may keep pushing changes until the deadline. The version that counts is **the one in the repository when it is collected at the deadline**: every entry is cloned right after 23:59 IST on 30 November, and the exact commit taken is recorded. Commit dates are not used. Keep the repository public until the results are out.

5. A run

- The course runs along +x and is 6 m wide. The robot starts at x = 1 m, standing in its default crouched pose, facing the finish.

- Every 0.02 s (50 Hz) your `act` receives the robot's sensors and returns **12 joint target angles**. The robot's position servos (stiffness 200 N·m/rad, torque limit 150 N·m) track them. Targets outside the joint limits are clipped; a non-finite value keeps that joint's previous target.
- A run ends when the robot:
  - **finishes**: its torso crosses the finish line;
  - **falls**: its torso touches the ground, comes within 0.3 m of the terrain under it, or tilts more than 60°;
  - **leaves the track**: its torso comes within 0.3 m of either side;
  - or reaches the **time limit of 180 s**.
- **A crashed controller doesn't stop the clock.** If `reset` or `act` raises an error, returns something other than 12 numbers, or takes longer than 5 s to answer, the controller is stopped and the robot holds its last targets until the run ends. `reset` itself may take up to 60 s.
- **Time budget:** each `act` call has **20 ms** of wall-clock time on one CPU core of the grading machine. A late answer is thrown away and the robot keeps its previous targets for that step, as a real robot would. The grader counts late steps; keep well under the budget.

The obstacles are built from these section types: flat, rough ground, stairs, hurdles, a ramp, a narrow bridge, waves, a trench, and barriers. Their parameters are in track.py.

What your controller can read

Your controller sees only what `reset` and `act` give it; everything else about the simulation is off limits (section 8). All readings are exact (no sensor noise).

Every step, in `act(obs)`:

READING	SIZE	WHAT IT IS
`time`	1	seconds since the start
`joint_pos`, `joint_vel`	12 + 12	joint angles [rad] and speeds [rad/s], legs in the order back-right, back-left, front-left, front-right
`gyro`	3	body turn rate [rad/s] (IMU)
`accelerometer`	3	body acceleration [m/s²], gravity included (IMU)
`gravity`	3	which way is down, in the body frame: `(0, 0, -1)` when level
`base_lin_vel`	3	body velocity [m/s], body frame, as a state estimator would give it
`position`	3	torso centre [m]: x along the track, y across it, z up, as motion capture would give it
`yaw`	1	heading [rad]; 0 faces the finish
`foot_contact`	4	which feet are touching the ground
`height_scan`	13 × 7	the terrain sensor (below)
`last_action`	12	the joint targets applied last step

The terrain sensor (`height_scan`) is how the robot sees the course. It is a grid of 13 × 7 terrain heights around the robot, from 0.8 m behind to 1.6 m ahead and 0.6 m to each side, turned with the robot's heading; each value is the terrain height there minus the torso's height (about −0.86 m on flat ground, less negative over a step or a hurdle ahead). Think of it as the height map a lidar or depth camera would build, already processed. **There is no camera image and no raw lidar point cloud.** The grid points' positions are in `info["scan_points"]`.

Once per run, in `reset(info)`: the control period `dt` (0.02 s), the standing pose `default_pose`, the joint limits `joint_lower` and `joint_upper`, `scan_points`, the finish line's position `finish_x`, the track width `track_width`, and `barriers`: for each barrier, where it is along and across the track, its height, its bonus and the centre of the open lane around it.

The track's layout is not given: not the sections, their order or their sizes. Your controller finds out what's ahead from the terrain sensor.

6. Barriers: jump or go around

At a barrier the track forks. A solid block covers one side of the track and the centreline; a lane about 2.5 m wide stays open on the other side. Your controller learns where each barrier is, how tall it is, which side is open and what bonus it gives from `info["barriers"]` in `reset`.

- **Over cleanly:** the torso passes over the barrier and **no part of the robot touches it** (its top, or any face above the ground). You earn the barrier's bonus: **5 s off your finish time** on the practice track.
- **Around:** walk through the open lane. No bonus, no penalty.
- **Touched:** climbing onto the barrier, kicking it, or clipping it with a leg on the way over doesn't end the run, but earns no bonus.

The route is decided where the torso passes the middle of the barrier. Bonuses count only if you finish.

7. Scoring

- Each entry runs **5 times** on the competition arena. The robot is the same in every run (no change to friction, mass or motor strength); only its start moves a little: up to 5 cm sideways and 3° in heading, always facing the finish, from seeds only the organiser knows.
- **Every entry gets the same 5 runs.** Run 1 starts from the same spot for everyone, and so on, so nobody gets luckier starts than anyone else.
- A run that **finishes** scores its **finish time minus its barrier bonuses**; lower is better.
- A run that **doesn't finish** scores the **distance** its torso got along the track; further is better.
- **Any finish beats any non-finish.**
- An entry's score is its **median run**: the 3rd best of the 5.
- **Lowest median score wins.** A tie is broken by the better average over all 5 runs, where a run that didn't finish counts as 180 s plus the distance it had left.

The competition arena

The arena is **secret** until the results. It has **exactly the same sections as the practice track** (tracks/practice.yaml), with the same sizes and the same single barrier, **in a different order**. A controller that handles every section of the practice track, whatever comes before or after it, is ready for the arena.

To check that, practise on reordered tracks: copy `practice.yaml`, shuffle its sections, and grade on your copy. The track generator makes other variations too:

```
python -m sprout_challenge.track --seed 7 --difficulty 0.8 --out my_track.yaml
```

A controller that counts on the order of the practice track (for example, "stairs come after 11 m") will fail on the arena; one that reads the terrain scan and `info["barriers"]` will not.

8. What's allowed

- **Any control method:** a hand-written gait, PID, model-predictive control, reinforcement learning, imitation, optimisation, or a mix. Train however and wherever you like; no GPU is needed.
- **Starting from the examples is fine,** including improving the baseline network. Otherwise the entry must be your own work.
- At grading time your code may use the **Python standard library, numpy, scipy, onnxruntime and this package**. Bundle anything else inside your folder, as pure Python. To use a network trained with PyTorch, JAX or similar, export it, for example to ONNX or to numpy arrays.
- There is **no network access** during grading, and memory is limited to 2 GB.

Not allowed (the entry is disqualified):

- Reading the simulator's state, or anything about the run, other than what `reset` and `act` receive (section 5).
- Interfering with the grader, its timing, its files, or other entries.
- Trying to find out the secret arena or seeds before the results.
- Entering someone else's work as your own, or more than one entry per person.

9. Grading

Every entry is graded the same way, on the organiser's machine, in the Docker image built from docker/Dockerfile: Python 3.11, no network, 1 CPU core, 2 GB of memory. You can run exactly the same container yourself, on the practice track or on any track you generate:

```
docker build -f docker/Dockerfile -t sprout-challenge .
docker run --rm --network none --cpus 1 --memory 2g -v "$PWD/my_entry":/submission/
my_entry:ro sprout-challenge \
    python -m sprout_challenge.grade /submission/my_entry --track practice
```

If your entry works with `python -m sprout_challenge.grade` on your computer but fails in the container, the container's result is the one that counts.

10. Prize

- **One $20 gift card** (Amazon, or an equivalent gift card available in the winner's country) for the entry with the best score on the competition arena. There are no runner-up prizes.

- The winner is announced on Instagram and contacted by DM. The gift card is sent within 14 days of the results, to an email address the winner gives by DM.
- If the winner doesn't reply within 14 days of being contacted, the prize goes to the next-best entry.
- The prize can't be exchanged for cash.
- The **top 10** entries on the leaderboard get a video of their run featured on @shantanu.builds.robots, credited to their Instagram handle.

11. Judging and disputes

- The organiser grades every entry with `scripts/grade_all.sh` on the arena and the secret seeds, and publishes the leaderboard, the arena and the seeds with the results, so anyone can re-run any entry and check its score.
- If a bug in the competition code turns out to change results, the organiser may fix it and re-grade every entry.
- If a rule is unclear, ask by DM before the deadline. Answers that change how a rule is read are added to this file for everyone.
- The organiser's decision is final.
- By entering you agree that your entry, your Instagram handle, your score and videos of your robot's runs may be published.

12. Instagram

This competition is not sponsored, endorsed or administered by, or associated with, Instagram or Meta. By entering, you release Instagram and Meta from any responsibility for it. Questions go to @shantanu.builds.robots, not to Instagram.